

Database Systems

Lecture 5 – Functional Dependencies & Normalization

Dr. Yatish Bathla, PhD. (Information Technology), Obuda University, Budapest

Learning outcomes and agenda

Time	Block
0-12	Why normalise; anomalies; lossless decomposition
12-27	Functional dependencies, axioms, closure
27-34	Worked examples: keys, canonical cover
34-50	Normal forms: 1NF, 2NF, BCNF
50-57	3NF synthesis; BCNF vs 3NF
57-60	Summary

By the end you can:

- Spot anomalies and fix them by decomposition
- Compute closure and candidate keys
- Test 1NF, 2NF, 3NF, BCNF
- Run the BCNF and 3NF algorithms
- Explain trade-offs

Goals of good relational design

ID	name	salary	dept_name	building	budget
22222	Einstein	95000	Physics	Watson	70000
83821	Brandt	92000	Comp. Sci.	**Taylor**	**100000**
45565	Katz	75000	Comp. Sci.	**Taylor**	**100000**

in_dep: Comp. Sci. building and budget are stored twice

No redundancy

Each fact is stored once

No anomalies

Update, insert, delete safely

Lossless

Decomposition must not lose information

Dependency preserving

Constraints stay checkable without joins

Normalization is the method that achieves these four goals.

Anomalies on one table

student_id	student_name	course_id	course_title	grade
S1	Asha	C1	DBMS	A
S1	Asha	C2	Networks	B
S2	Ravi	C1	DBMS	B

Update anomaly

Rename C1 to "Database Systems": must change two rows. Change one and the table is inconsistent.

Insertion anomaly

New course C3 with no students yet: no row can hold it without NULL student_id, which is part of the key.

Deletion anomaly

Delete (S1, C2) when S1 drops Networks: the fact that C2 is called Networks vanishes.

Lossless decomposition

employee(ID, name, street, city, salary)

ID	name	street	city	salary
57766	Kim	Main	Perryridge	75000
98776	Kim	North	Hampton	67000

Split into employee1(ID, name) and employee2(name, street, city, salary), then join on name:

ID	name	street	city	salary
57766	Kim	Main	Perryridge	75000
57766	Kim	**North**	**Hampton**	**67000**
98776	Kim	**Main**	**Perryridge**	**75000**
98776	Kim	North	Hampton	67000

Lossy!

- 2 rows became 4
- Red rows are spurious: they never existed
- name is not unique, so it cannot link the pieces

Lossless if R1 ∩ R2 → R1 or R1 ∩ R2 → R2

Functional dependency: definition

A	B	C	D
a1	b1	c1	d1
a1	b2	c1	d2
a2	b2	c2	d2
a2	b3	c2	d3
a3	b3	c2	d4

A → C holds: same A always gives the same C.

C → A fails: c2 appears with a2 and a3.

α → β

- α and β are sets of attributes
- If two rows agree on α, they must agree on β
- Read: α determines β
- Trivial if β ⊆ α (for example AB → A)
- An FD is a rule of the real world; the data can only violate it, never prove it

Functional dependencies and keys

Superkey

K is a superkey of R if K → R: K determines every attribute

Candidate key

A superkey with no unnecessary attribute (minimal)

Trivial FD

α → β with β ⊆ α. Always true; ignore it

Prime attribute

Belongs to at least one candidate key

Relation	FDs	Candidate key
instructor(ID, name, salary, dept_name)	ID → name, salary, dept_name	ID
department(dept_name, building, budget)	dept_name → building, budget	dept_name
takes(ID, course_id, sec_id, grade)	ID, course_id, sec_id → grade	ID, course_id, sec_id

Finding FDs from business rules

Business rule	Functional dependency	Valid?
A student ID identifies one student	student_id → student_name	Yes
A course has one title	course_id → course_title	Yes
A student gets one grade per course section	student_id, course_id, sec_id → grade	Yes
A postal code belongs to one city	postal_code → city	Yes (typically)
Names identify people	name → student_id	**No: names repeat**
A city has one postal code	city → postal_code	**No: cities have many**

Ask: "Can two rows with the same left side have different right sides?" If yes, no FD.

Armstrong's axioms and derived rules

Rule	Statement	Example
Reflexivity	If $\beta \subseteq \alpha$ then $\alpha \rightarrow \beta$	AB → A
Augmentation	If $\alpha \rightarrow \beta$ then $\gamma\alpha \rightarrow \gamma\beta$	A → B gives AC → BC
Transitivity	If $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$ then $\alpha \rightarrow \gamma$	ID→dept, dept→building gives ID→building
Union	If $\alpha \rightarrow \beta$ and $\alpha \rightarrow \gamma$ then $\alpha \rightarrow \beta\gamma$	A → B, A → C gives A → BC
Decomposition	If $\alpha \rightarrow \beta\gamma$ then $\alpha \rightarrow \beta$ and $\alpha \rightarrow \gamma$	A → BC gives A → B
Pseudotransitivity	If $\alpha \rightarrow \beta$ and $\gamma\beta \rightarrow \delta$ then $\alpha\gamma \rightarrow \delta$	A → B, CB → D gives AC → D

The first three axioms are sound and complete; the rest follow from them.

Closure of an attribute set

```
result := a;
repeat
   for each FD  b -> g  in F do
      if b is a subset of result
         then result := result U g
until result does not change

/* result is a+ */
```

What α+ gives us

- Superkey test: α is a superkey if α+ contains all of R
- FD test: α → β holds if β ⊆ α+
- Candidate keys: find minimal α with α+ = R

Example: F = {ID → dept_name, dept_name → building} so (ID)+ = {ID, dept_name, building}

Worked example: closure & key

R = (A, B, C, G, H, I) **F = { A→B, A→C, CG→H, CG→I, B→H }**

Compute (AG)+

Step	FD used	result
0	start	A G
1	A → B	A B G
2	A → C	A B C G
3	CG → H	A B C G H
4	CG → I	A B C G H I

Conclusions

- (AG)+ = R, so AG is a superkey
- A+ = {A, B, C, H}: missing G, I
- G+ = {G}: missing most
- So AG is minimal: a candidate key
- A and G never appear on a right side, so they must be in every key

Shortcut: attributes that never appear on any right side must be in every candidate key.

Canonical cover

Step	Rule applied	Result
0	Given F	A → BC, B → C, A → B, AB → C
1	Union: A → BC and A → B	A → BC, B → C, AB → C
2	A is extraneous in AB → C because B → C	A → BC, B → C
3	C is extraneous in A → BC because A → B, B → C	**A → B, B → C**

Extraneous attribute

Can be removed from an FD without changing what F implies

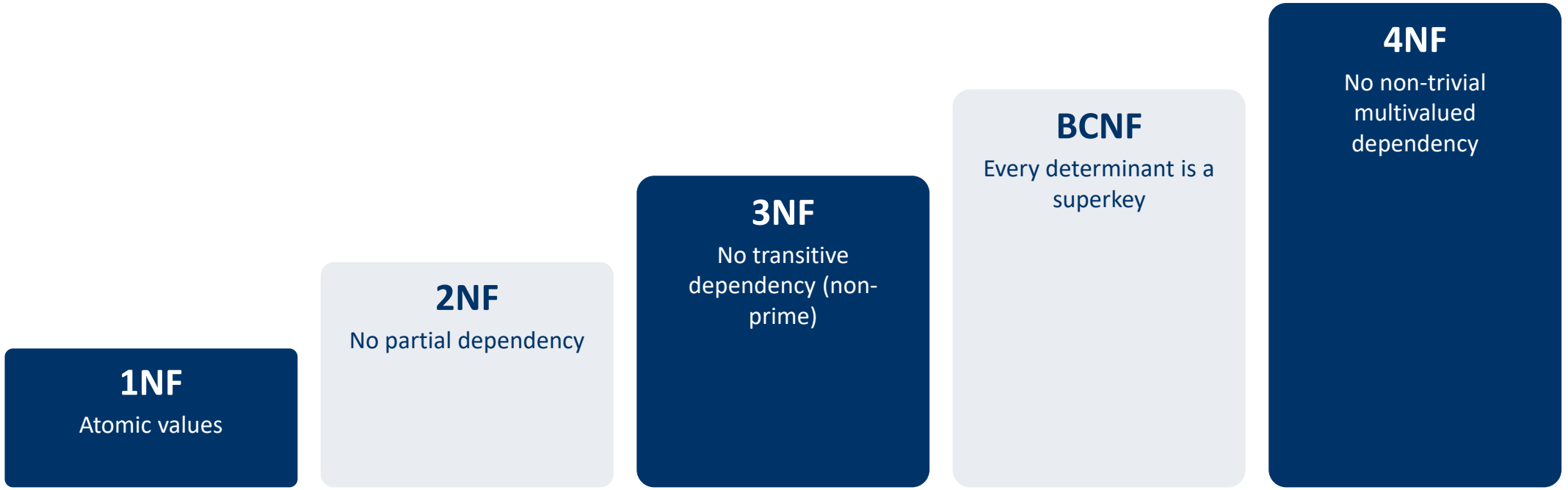
Canonical cover Fc

Equivalent to F, no extraneous attributes, each left side unique

Why

Smallest set to enforce; input to the 3NF algorithm

The normal form ladder



Each step removes one more kind of redundancy. Most practical designs target 3NF or BCNF.

First normal form: atomic values

Not in 1NF

ID	name	phones
S1	Asha	98100, 98101
S2	Ravi	98102

Rule

- A value is atomic if the DBMS treats it as indivisible
- Sets, lists and repeating groups violate 1NF
- Mapping rule from Lecture 1: multivalued attribute becomes its own table

In 1NF (two tables)

ID	name
S1	Asha
S2	Ravi

ID	phone
S1	98100
S1	98101
S2	98102

2NF: partial dependency

student_id	course_id	student_name	grade
S1	C1	Asha	A
S1	C2	Asha	B
S2	C1	Ravi	B

Key: (student_id, course_id)

student_id → student_name is a partial dependency: it depends on only part of the key.

(student_id, course_id) → grade is a full dependency.

Decompose

student(student_id, student_name)

enrol(student_id, course_id, grade)

2NF: in 1NF and no non-prime attribute depends on part of a candidate key.

Boyce-Codd normal form (BCNF)

Definition

- For every nontrivial FD α → β that holds on R, α must be a superkey of R
- To test R, checking the FDs in F is enough

FD in inst_dep	Left side a superkey?	Verdict
ID → name, salary, dept_name	Yes (ID)	**OK**
dept_name → building, budget	No	**Violation**

inst_dep(ID, name, salary, dept_name, building, budget)

instructor

ID, name, salary, dept_name

department

dept_name, building, budget

Lossless: the common attribute dept_name is the key of department.

BCNF decomposition algorithm

```
result := { R };
while (some Ri in result is not in BCNF) do
   choose a nontrivial FD  a -> b  on Ri
      with a -> Ri not in F+ and a ∩ b = {}
   result := (result - Ri) U (Ri - b) U (a, b)
```

Example: dept_advisor(s_ID, i_ID, dept_name)

F = { i_ID → dept_name, s_ID dept_name → i_ID }

i_ID is not a superkey, so split on i_ID → dept_name:

(s_ID, i_ID) and (i_ID, dept_name)

Warning

- The result is lossless
- But s_ID, dept_name → i_ID can no longer be checked in one table
- BCNF cannot always preserve dependencies

This is the reason 3NF exists.

3NF and the synthesis algorithm

Definition

- For every nontrivial FD α → β:
- (1) α is a superkey, or
- (2) every attribute in β − α is in a candidate key

```
1. Compute a canonical cover Fc
2. For each FD a -> b in Fc:
      create schema (a, b)
3. If no schema holds a candidate key,
      add a schema with one
4. Remove schemas contained in others
```

registration(student_id, student_name, course_id, course_title, grade)

Step	Result
FDs	sid → sname; cid → ctitle; sid cid → grade
Key	(student_id, course_id)
R1	(student_id, student_name)
R2	(course_id, course_title)
R3	(student_id, course_id, grade)

R3 holds the candidate key; all three schemas are in BCNF too.

BCNF versus 3NF, and beyond

Property	BCNF	3NF
Removes FD-based redundancy	Completely	Almost (small exception)
Lossless join	Always	Always
Dependency preserving	Not always	Always
Algorithm	Decompose violating FDs	Synthesis from canonical cover
Typical choice	When all FDs can be kept	When BCNF would lose a dependency

4NF

Removes redundancy from multivalued dependencies

Denormalization

Duplicate data on purpose for speed; accept update cost

Design practice

Start from an ER model, then check normal forms

Summary and review questions

Problem

Redundancy causes anomalies; decompose to fix

Tool

FDs, Armstrong axioms, closure, canonical cover

Standards

1NF, 2NF, 3NF, BCNF (4NF)

Method

Find keys, test FDs, decompose losslessly

Review questions

- R(A,B,C,D), F = {A→C, C→D}: find (AB)+ and a key
- R(ID, name, dept, building), ID→name,dept; dept→building: which NF?
- Why is a split on a non-unique column lossy?

Next: Laboratory – ER design, normalization practice, visualization, probability in code